การใช้ SubQuery ใน SQL

Woottikarn Hongwiengchan

SubQuery คืออะไร

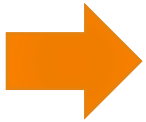
SubQuery (หรือ Nested Query) คือคำสั่ง SELECT ที่อยู่ภายในคำสั่ง SQL อีกทีหนึ่ง ใช้เพื่อดึงข้อมูลมาช่วยในการกรองข้อมูลหลัก

คำสั่งที่ใช่ร่วมกับ SubQuery

คำสั่ง	ใช้เมื่อไร
SELECT	ใช้ SubQuery เพื่อคัดกรอง, คำนวณ หรือสร้างข้อมูลเสริม
WHERE	ใช้ SubQuery เพื่อสร้างเงื่อนไข เช่น WHERE column IN (...)
FROM	ใช้ SubQuery แทนตาราง (เรียกว่า Inline View)
HAVING	ใช้ SubQuery เพื่อเปรียบเทียบค่าหลัง GROUP BY
IN, NOT IN	ใช้ SubQuery เพื่อกำหนดชุดของค่าเปรียบเทียบ
EXISTS, NOT EXISTS	ใช้ SubQuery เพื่อตรวจสอบการมีอยู่ของข้อมูล
UPDATE, DELETE	ใช้ SubQuery เพื่อเลือกแถวที่ต้องการแก้ไขหรือลบ
Aggregate Functions	ใช้ SubQuery เพื่อเปรียบเทียบค่าข้อมูลที่ผ่านการบวนการคำนวน เช่น (AVG,Count,Sum)

ตัวอย่างที่ 1:

```
SELECT CustomerName, Country
FROM Customers
WHERE Country IN (
    SELECT Country
    FROM Customers
    GROUP BY Country
    HAVING COUNT(*) > 3
);
```



แสดง ชื่อลูกค้า (CustomerName) ที่อาศัยอยู่ในประเทศที่มีลูกค้าอยู่มากกว่า 3 คน

1

```
SELECT Country
    FROM Customers
    GROUP BY Country
    HAVING COUNT(*) > 3
```

2

```
SELECT CustomerName, Country
FROM Customers
WHERE Country IN (.....)
```

ตัวอย่างที่ 2:

หาสินค้าที่มีราคาสูงกว่าค่าเฉลี่ยของราคาสินค้าทั้งหมด

```
SELECT ProductID, ProductName
FROM Products
WHERE Price > (
    SELECT AVG(Price)
    FROM Products
);
```

อธิบายตัวอย่างที่ 2

```
SELECT AVG(Price)
  FROM Products
```

SubQuery ชั้นใน:

→ จะแสดงค่าเฉลี่ยราคาสินค้าทั้งหมดออกมา

อธิบายตัวอย่างที่ 2

```
SELECT ProductID, ProductName
FROM Products
WHERE Price > (.............)
```

SubQuery ตัวหลัก:

→ จะแสดงข้อมูลสินค้าที่มีราคาสูงเกินเงื่อนไขที่กำหนด

ตัวอย่างที่ 3:

หาชื่อสินค้า (ProductName) ที่มียอดขายรวมสูงกว่า 'Chang'

```
SELECT p.ProductName, SUM(o.Quantity * p.Price) AS TotalSales
FROM OrderDetails o
INNER JOIN Products p ON o.ProductID = p.ProductID
GROUP BY p.ProductName
HAVING SUM(o.Quantity * p.Price) > (
    SELECT SUM(od.Quantity * p.Price)
    FROM OrderDetails od
    INNER JOIN Products p ON od.ProductID = p.ProductID
    WHERE p.ProductName = 'Chang'
);
```

อธิบายตัวอย่างที่ 3

```
SELECT SUM(od.Quantity * p.Price)
FROM OrderDetails od
INNER JOIN Products p ON od.ProductID = p.ProductID
WHERE p.ProductName = 'Chang'
```

SubQuery ชั้นใน:

→ หายอดขายของสินค้าที่ชื่อว่า 'Chang'

อธิบายตัวอย่างที่ 3

```
SELECT p.ProductName, SUM(o.Quantity * p.Price) AS
TotalSales
FROM OrderDetails o
INNER JOIN Products p ON o.ProductID = p.ProductID
GROUP BY p.ProductName
HAVING SUM(o.Quantity * p.Price) > (.....)
```

SubQuery ตัวหลัก:

→ จะแสดงข้อมูลสินค้าที่มียอดขายมากกว่าเงื่อนไขตามที่กำหนด

ตัวอย่างที่ 4:

หาข้อมูลคำสั่งซื้อจากลูกค้าที่ไม่เคยสั่งสินค้าประเภทเครื่องดื่ม

```
SELECT * FROM Orders
WHERE CustomerID IN (
   SELECT CustomerID
   FROM Customers
   WHERE CustomerID NOT IN (
            SELECT DISTINCT o.CustomerID
            FROM ((Orders o
            INNER JOIN OrderDetails od ON o.OrderID = od.OrderID)
            INNER JOIN Products p ON od.ProductID = p.ProductID)
            INNER JOIN Categories c ON p.CategoryID = c.CategoryID
            WHERE c.CategoryName = 'Beverages';
            )
);
```

อธิบายตัวอย่างที่ 4

```
SELECT DISTINCT o.CustomerID
FROM ((Orders o
INNER JOIN OrderDetails od ON o.OrderID = od.OrderID)
INNER JOIN Products p ON od.ProductID = p.ProductID)
INNER JOIN Categories c ON p.CategoryID = c.CategoryID
WHERE c.CategoryName = 'Beverages';
```

SubQuery ชั้นในสุด:

→ หารายชื่อลูกค้าไม่ซ้ำกันที่เคยสั่งสินค้า **ประเภทเครื่องดื่ม**

อธิบายตัวอย่างที่ 4

```
SELECT CustomerID
FROM Customers
WHERE CustomerID NOT IN (...)
```

SubQuery ชั้นกลาง:

→ กรองเฉพาะลูกค้าที่ ไม่อยู่ในกลุ่มนั้น

อธิบายตัวอย่างที่ 4

```
SELECT * FROM Orders
WHERE CustomerID IN (...)
```

SubQuery คำสั่งหลัก:

→ หาคำสั่งซื้อของลูกค้ากลุ่มนั้น

ข้อควรระวังในการใช้ SubQuery

- หลีกเลี่ยงการใช้มากเกินไป อาจทำให้ช้า

- ตรวจสอบว่าเหมาะสมกับสถานการณ์

- ระวังเรื่อง NULL และผลลัพธ์ว่าง

- สามารถเขียนซ้อนกันหลายระดับได้ เช่น SELECT ซ้อน SELECT ซ้อน SELECT แต่ควรระวังเรื่องประสิทธิภาพ

อธิบาย EXISTS

- ใช้ตรวจสอบว่ามีข้อมูลตามเงื่อนไขใน SubQuery หรือไม่ (ผลลัพธ์เป็น TRUE/FALSE)
- คำสั่ง **WHERE o.CustomerID = c.CustomerID** สามารถใช้ได้โดยไม่ต้องใช้ **JOIN** โดยตรง ถ้าอยู่ภายใน **Subquery** อย่างในคำสั่ง **EXISTS, IN,** หรือ **ANY/ALL** เช่นในกรณีนี้:

  c คือ **alias** ของตารางหลัก **(Customers)**

  o คือ **alias** ของตารางภายใน **Subquery (Orders)**

การเขียนเงื่อนไขเช่น **o.CustomerID = c.CustomerID** จึงเป็นการเชื่อมโยงแบบเชิงตรรกะ (**logical correlation)** ระหว่างตารางหลักและตารางภายใน **Subquery**

ตัวอย่างที่ 5:

```
SELECT CustomerName
FROM Customers c
WHERE EXISTS (
        SELECT *
        FROM Orders o
        WHERE o.CustomerID = c.CustomerID
        AND year(o.OrderDate) = 1997
);
```

หาลูกค้าที่เคยสั่งสินค้าในปี 1997

ตัวอย่างที่ 6:

```
SELECT ProductName, Price
FROM Products
WHERE Price > ANY (
  SELECT Price
  FROM Products
  WHERE CategoryID = (
    SELECT CategoryID
    FROM Categories
    WHERE CategoryName = 'Beverages'
  )
);
```

หาสินค้าที่ราคามากกว่าสินค้าตัวใดตัวหนึ่ง ในหมวด Beverages

ตัวอย่างที่ 6:

```
SELECT ProductName, Price
FROM Products
WHERE Price > ALL (
  SELECT Price
  FROM Products
  WHERE CategoryID = (
    SELECT CategoryID
    FROM Categories
    WHERE CategoryName = 'Produce'
  )
);
```

หาสินค้าที่มีราคามากกว่าสินค้าทุกตัวในหมวด Produce